5.2 Agentowe Przeglądy Kodu (Code Review): Implementacja botów AI analizujących Pull Requesty pod kątem bugów i stylu

Moduł 5: Agenty AI w procesach CI/CD i DevOps

Krzysztof Rytlewski · 2026

Czym jest agentowy code review i dlaczego warto go stosować?

Code review to praktyka analizy zmian przed scaleniem z gałęzią główną. Tradycyjny przegląd jest wartościowy, ale ma ograniczenia: jest czasochłonny, zależy od dostępności recenzenta, a rutynowe błędy pochłaniają czas, który można poświęcić na merytoryczną dyskusję.

Bot AI do code review automatyzuje rutynową część przeglądu:

1. Wykrywa zdarzenie otworzenia lub aktualizacji Pull Requesta.
2. Pobiera diff (zestaw zmian) z GitHub API.
3. Przesyła diff do LLM z odpowiednim promptem.
4. Publikuje komentarze w PR — ogólne i liniowe (wskazujące konkretną linię).

Bot nie zastępuje ludzkiego przeglądu — uzupełnia go, eliminując rutynowe problemy i pozwalając recenzentom skupić się na architekturze i logice biznesowej.

Anatomia diff'a PR — format unified diff

GitHub API udostępnia diff w formacie unified diff — standardowym formacie tekstowym opisującym zmiany linia po linii.

```
diff --git a/backend/routes/chat.js b/backend/routes/chat.js
--- a/backend/routes/chat.js
+++ b/backend/routes/chat.js
@@ -12,6 +12,10 @@ router.post('/', requireAuth, async (req, res) => {
   const { message } = req.body;
+  // TODO: usunąć przed produkcją
+  console.log('DEBUG klucz:', process.env.OPENAI_API_KEY);
```

Linie zaczynające się od `+` to dodane linie, od `-` to usunięte. Sekcja `@@` wskazuje numer linii. Dla LLM dostarczamy pełny diff, kontekst projektu i instrukcje co sprawdzać.

Pobieranie diff'a przez GitHub API

```
const { data: files } = await octokit.pulls.listFiles({
  owner, repo, pull_number: prNumber,
});

// files zawiera: filename, patch (diff), status
const diffSummary = files.map(f =>
  `### f.filename({f.status})\n\`\`\`diff\n${f.patch ?? '(brak diff)'}\n\`\`\``
).join('\n\n');
```

Przed wysłaniem do LLM filtrujemy pliki — analizujemy tylko `.js` i `.ts`, pomijamy `node_modules`, pliki `dist` i lock files. Ogranicza to liczbę tokenów i koncentruje przegląd na istotnych zmianach.

Architektura bota — GitHub Actions + Node.js + OpenAI

```yaml
# .github/workflows/ai-review.yml
on:
  pull_request:
    types: [opened, synchronize]

permissions:
  contents: read
  pull-requests: write   # niezbędne do publikowania komentarzy

jobs:
  review:
    runs-on: ubuntu-latest
    if: github.event.pull_request.head.repo.full_name == github.repository
    steps:
      - uses: actions/checkout@v4
```

Warunek `if` blokuje uruchamianie dla PR z zewnętrznych forków, które nie mają dostępu do sekretów repozytorium.

System prompt i żądanie do LLM

Skuteczny system prompt definiuje rolę bota i zakres przeglądu. Bot analizuje zmiany pod kątem pięciu kategorii:

1. Błędów bezpieczeństwa (wstrzyknięcia, odsłonięte sekrety, brak walidacji wejścia).
2. Błędów logicznych i potencjalnych wyjątków runtime.
3. Problemów z wydajnością (N+1 queries, brakujące indeksy).
4. Niespójności stylu z resztą kodu.
5. Brakującej obsługi błędów (`try/catch`, walidacja `req.body`).

LLM odpowiada w formacie JSON: `{ "summary": "...", "issues": [{ "file": "...", "line": N, "severity": "error|warning|info", "comment": "..." }] }` — dzięki `response_format: { type: 'json_object' }` odpowiedź jest zawsze parsowalna.

Publikowanie komentarza w PR

Po otrzymaniu odpowiedzi od LLM bot publikuje ogólny komentarz na poziomie PR przez `octokit.issues.createComment()`. Komentarz zawiera podsumowanie i listę wszystkich wykrytych problemów z podaniem pliku, numeru linii i poziomu istotności.

Jeśli LLM nie wykryło problemów, komentarz zawiera pozytywne podsumowanie i znacznik ✅. Jeśli wykryto problemy, każda uwaga wyświetlana jest z nagłówkiem `**ERROR|WARNING|INFO**`, nazwą pliku i numerem linii — recenzent od razu wie, gdzie szukać problemu.

Komentarze liniowe — przypisanie uwagi do konkretnej linii

GitHub API pozwala publikować komentarze bezpośrednio przy danej linii kodu w diff'ie — są bardziej przydatne niż ogólny komentarz.

```
const { data: pr } = await octokit.pulls.get({ owner, repo, pull_number: prNumber });
const commitId = pr.head.sha;

await octokit.pulls.createReviewComment({
  owner, repo, pull_number: prNumber,
  commit_id: commitId,
  path: 'backend/routes/chat.js',
  line: 15,      // numer linii w nowym pliku
  side: 'RIGHT', // RIGHT = nowa wersja pliku
  body: '⚠️ Klucz API logowany do konsoli. Usuń przed mergem.',
});
```

Recenzent widzi uwagę dokładnie przy problematycznym kodzie, co skraca czas analizy.

Poziomy istotności i blokowanie merge

Poziom	Znaczenie	Akcja bota
`error`	Błąd bezpieczeństwa lub logiczny — blokuje merge	Komentarz liniowy + `REQUEST_CHANGES`
`warning`	Potencjalny problem — warto rozważyć	Komentarz liniowy + `COMMENT`
`info`	Sugestia stylistyczna	Ogólny komentarz PR

Bot może automatycznie zablokować merge przez wystawienie review statusu `REQUEST_CHANGES` przy wykryciu problemów poziomu `error`:

```
await octokit.pulls.createReview({
  owner, repo, pull_number: prNumber, commit_id: commitId,
  event: hasErrors ? 'REQUEST_CHANGES' : 'COMMENT',
  body: 'Automatyczny przegląd AI zakończony.',
});
```

Ograniczenia i dobre praktyki agentowego code review

Mocne strony: wykrywanie wzorców bezpieczeństwa, egzekwowanie stylu, dostępność 24/7, brak zmęczenia przy dużych PR.

Ograniczenia:

- LLM może generować **fałszywe alarmy** — szczególnie przy złożonej logice biznesowej.
- Brak pełnego kontekstu projektu — bot widzi tylko diff, nie całą bazę kodu.
- Kosztowne przy bardzo dużych PR (duży diff = dużo tokenów).

Dobre praktyki:

- Ogranicz przegląd do rozsądnego rozmiaru diff'a (max ~500 linii na wywołanie).
- Dodaj do promptu konwencje projektu (np. „używamy `async/await`, nie `.then()`").
- Daj deweloperom możliwość pominięcia bota przez etykietę `skip-ai-review` na PR.
- Archiwizuj wyniki przeglądów w MongoDB do późniejszej analizy jakości bota.